

ECE 473/573
Cloud Computing and Cloud Native Systems
Lecture 13 Web Security

Professor Jia Wang
School of Engineering, Applied Science, and Health
Illinois Institute of Technology

September 30, 2026

Outline

Web Security

JSON Web Token (JWT)

Reading Assignment

- ▶ This lecture: Web Security
- ▶ Next two lectures: Distributed database systems
 - ▶ Cassandra - A Decentralized Structured Storage System
<https://www.cs.cornell.edu/projects/ladis2009/papers/lakshman-ladis2009.pdf>
 - ▶ Spanner: Google's Globally-Distributed Database
http://static.googleusercontent.com/external_content/untrusted_dlcp/research.google.com/en/archive/spanner-osdi2012.pdf

Outline

Web Security

JSON Web Token (JWT)

HyperText Transfer Protocol Secure (HTTPS)

- ▶ A.k.a. HTTP over SSL or HTTP over TLS.
 - ▶ HTTP communication entirely on top of TLS (over TCP), usually use port 443.
 - ▶ Provide confidentiality and integrity.
 - ▶ Usually server authentication only, but client authentication could also be added.
- ▶ Domain name authentication
 - ▶ HTTPS server certificates need to include matching domain names and/or ip addresses for the connection to be considered secure by browsers.
 - ▶ Provide protection against IP address spoofing and DNS spoofing.
 - ▶ CA certificates can also be included with new browser installations – don't install browser from unknown sources!

Authentication for RESTful Services

- ▶ A web session includes many requests/responses.
 - ▶ Usually, RESTful requests are protected by HTTPS with server authentication only.
 - ▶ Clients are authenticated with other means like username and password.
- ▶ It is not practical to send a username and password for every RESTful request.
 - ▶ Users expect to enter their credentials once per login.
 - ▶ Reusing a password on every request increases its exposure.
- ▶ How can a client prove its identity on later requests without sending the password again?

Session Authentication with Cookies

- ▶ Cookie: an HTTP mechanism for storing client-side state.
- ▶ A client starts the session by sending credentials to a login service.
- ▶ The service authenticates the user and sends back a cookie containing a randomly generated string, which is usually known as the access token.
- ▶ The client, usually a browser, automatically sends matching cookies according to their scope and policy hereafter.

Threats to Cookies

- ▶ Guessing or fixing an access token.
 - ▶ Attackers may trick the server into using a known token.
 - ▶ Attackers may guess a weak token.
 - ▶ Always generate a fresh, unpredictable token after login.
- ▶ Attackers may read a token in transit.
 - ▶ HTTPS should be used to protect all HTTP communications.
- ▶ Attackers may steal a token from the client.
 - ▶ Enforce a server-side session lifetime.
 - ▶ Use cookie policy to reduce specific risks.
 - ▶ Keep tokens out of URLs, logs, and screenshots.
- ▶ There are other concerns like Cross-Site Scripting (XSS) and Cross-Site Request Forgery (CSRF).

Outline

Web Security

JSON Web Token (JWT)

Motivation

- ▶ How can microservices verify randomly generated tokens?
 - ▶ A login service stores the tokens to a database together with the corresponding users and expiry times.
 - ▶ Other services consult that database.
- ▶ Token validation on each request adds a database lookup.
 - ▶ Caching can reduce lookups but makes revocation less immediate.
- ▶ Can a service verify a token locally, without a database lookup?

JSON Web Token (JWT)

- ▶ JSON Web Token (JWT): RFC 7519, 2015
 - ▶ A compact format for claims.
 - ▶ A signed JWT carries claims and a verifiable signature.
- ▶ Signed JWTs can serve as the access tokens.
 - ▶ Its claims can include subject, issuer, audience, role, and expiry.
 - ▶ A verifier checks the signature, accepted algorithm, issuer, audience, and expiry. It need not query a database.
 - ▶ A signed payload is encoded, not secret. Token theft still matters.

JWT Message and Token Structure

- ▶ Header: {"alg": "RS256", "typ": "JWT"}
 - ▶ alg: signature algorithm
 - ▶ typ: message type, usually JWT
- ▶ Payload: {"sub": "admin", "iat": 1759276800, "exp": 1759280400}
 - ▶ Include claims (data) to be signed.
 - ▶ sub: subject, such as an account ID.
 - ▶ iat: issued at time, when the JWT token is generated
 - ▶ exp: expiration time, after which the JWT must be rejected
 - ▶ Both iat and exp are in Unix time, i.e. number of seconds since 01/01/1970 00:00:00 UTC.
- ▶ Signed JWT token: xxxxxxxx.yyyyyyy.zzzzzz
 - ▶ xxxxxxxx: Base64url encoding of header
 - ▶ yyyyyyy: Base64url encoding of payload
 - ▶ zzzzzz: Base64url encoding of the signature over xxxxxxxx.yyyyyyy.
 - ▶ While the encoding makes the compact string URL-safe, don't put tokens in URLs as they can leak through logs and history.

JWT Signature Algorithms

- ▶ With symmetric cryptography, a MAC algorithm is used.
 - ▶ E.g. "HS256" – use HMAC algorithm with SHA-256 hash.
 - ▶ Need a pre-shared secret among all services.
 - ▶ Every verifier holding the secret can also issue tokens.
 - ▶ Sharing the signing secret across many services broadens its exposure.
- ▶ With public-key cryptography, a digital signature is used.
 - ▶ E.g. "RS256" uses RSA with SHA-256.
 - ▶ Need to deploy public keys of services issuing tokens (e.g. the login service) to those verifying tokens (e.g. other services).
 - ▶ A verifier with only the public key cannot issue tokens.
 - ▶ Verifiers must obtain the correct public key through a trusted channel.

Authorization Header

- ▶ Instead of using cookies, RESTful clients can send tokens for each protected request explicitly in the request header as bearer tokens, e.g. `Authorization: Bearer <JWT>`.
- ▶ RESTful servers extract tokens from headers and verify them.
 - ▶ A valid signature alone is insufficient.
 - ▶ Validate issuer `iss` and audience `aud`.
 - ▶ Validate expiry `exp`.
 - ▶ Extract subject `sub` (and additional fields like `role`) for authorizations.

Authorizations

- ▶ Authorization: for this RESTful request, does the subject have the permission to access the resource?
 - ▶ This cannot be answered by JWTs alone and remains the responsibility of RESTful services.
- ▶ For example, in Project 3,
 - ▶ Add credits: admin role only.
 - ▶ Create a listing: subject sub must equal `seller_id`
 - ▶ Order a listing: subject sub must equal `buyer_id`

Summary

- ▶ Cookies and authorization headers are different ways to send bearer tokens on HTTP requests.
- ▶ Signed JWTs let services verify claims locally, but do not prevent token theft or replace per-route authorization.